

Build-Ready Requirements: ETF Research and Paper-Portfolio Prototype

1. Executive perspective

Product mission: build a local, single-user, browser-based workbench that ingests prior-day ETF and economic data, executes reproducible analytical rules, presents signals with evidence, and maintains a complete hypothetical portfolio. A signal is a **research artifact**, never an instruction or an executable order. Only the user may create or confirm a paper action, and the system must contain **no brokerage connector, brokerage credential model, real-order endpoint, or code path capable of transmitting an order**.

This specification expands the useful concepts already documented in [etf_analytics_platform_modernization_documentation_package_20260820_163532.pdf](#): configurable ETFs, validated daily OHLCV history, versioned analytics, evidence-bearing signals, provenance, PostgreSQL, and agent-readable documentation. It intentionally changes two earlier recommendations: the prototype will use local Kubernetes rather than Azure Container Apps, and **Tiingo Starter is no longer suitable for persistent prototype storage** because Tiingo's August 5, 2026 terms prohibit Starter-plan users from retaining Tiingo data in databases, files, logs, backups, or other durable storage. [\[etf_analyt...820_163532 | PDF\]](#) [\[api.tiingo.com\]](#), [\[etf_analyt...820_163532 | PDF\]](#)

2. Scope, assumptions, and non-goals

Mandatory scope

- User-defined ETF watchlists only; no automatic “all ETFs” universe.
- Prior-day/end-of-day ingestion, historical backfill, economic-series ingestion, deterministic analytics, research signals, backtesting, and full paper-portfolio accounting.
- Single local user initially; authentication may be a local access gate, but authorization boundaries and audit fields must remain explicit.
- PostgreSQL is the authoritative operational store.
- Modern TypeScript and Python; Rust requires an approved benchmark-backed ADR.
- Useful legacy concepts are preserved through mappings and characterization tests, not compatibility layers.

Explicit non-goals

Real-time streaming; intraday strategy evaluation; personalized investment advice; guaranteed returns; real-money execution; brokerage integration; paid data; options, futures, leverage, margin, short selling, tax reporting, production-scale multi-tenancy, HA/DR, mobile-native applications, Azure resources, and backward API/schema compatibility.

Key constraints

- Timestamps are stored in UTC; trading dates retain exchange-calendar meaning; the UI displays the configured local zone and UTC on demand.
 - Missing or unvalidated data **blocks dependent analytics** rather than silently substituting values.
 - Historical results are always labeled *hypothetical*, with assumptions and data versions visible.
 - Market-provider terms must be revalidated before first use and recorded as a provider-policy artifact.
-

3. Personas and principal journeys

Persona	Journey
Research user	Manage watchlists → inspect freshness → run or review analytics → inspect signal evidence → define hypothetical criteria → explicitly initiate a paper order.
Portfolio user	Configure starting cash → submit/confirm paper orders → review fills, lots, cash, positions, P&L, performance, benchmark, and reconciliations.
Operator/developer	Configure providers → run/backfill jobs → inspect gaps/errors → retry or replay → export diagnostics → back up/restore PostgreSQL.
Rule author	Inspect legacy rule behavior → define parameterized rule version → validate → run historical evaluation → publish for research use.
Build agent	Inspect legacy artifacts → maintain traceability → implement requirements → produce tests, contracts, migrations, runbooks, and ADRs.

4. Functional requirements

Priority: P0 = required working prototype; P1 = should-have; P2 = future seam.

ID	Priority	Requirement
----	----------	-------------

FR-WL-01	P0	Create, rename, archive, and list watchlists; add/remove/reorder ETFs; prevent duplicate active entries.
FR-WL-02	P0	Adding an ETF validates the symbol through the configured provider and starts an observable, resumable historical backfill.
FR-MD-01	P0	Store raw and normalized EOD OHLCV, adjustment mode, source, requested range, retrieval time, payload hash, run ID, and revision.
FR-MD-02	P0	Incremental refresh shall target expected completed sessions only and be idempotent by instrument, trading date, provider, adjustment policy, and revision.
FR-MD-03	P0	Detect gaps, duplicates, impossible OHLC relationships, negative prices/volume, unexpected nulls, stale symbols, and cross-run changes.
FR-MD-04	P0	Source failure or incomplete validation shall create a visible failed/partial state and suppress affected signals.
FR-EC-01	P0	Configure generic economic series without hard-coding an exhaustive indicator catalog.
FR-EC-02	P0	Persist series metadata, observation date, value, units, frequency, release/availability timestamp, source, retrieval time, and vintage/revision where supported.
FR-AN-01	P0	Execute versioned, parameterized deterministic rules against an immutable input-data snapshot.
FR-AN-02	P0	Each signal shall contain instrument/date, rule version, parameters, input range/hash, outcome, explanation, evidence values, warnings, and run provenance.
FR-AN-03	P0	Signals shall use neutral labels such as condition_met, condition_not_met, or legacy-derived names—not “buy now” or personalized recommendations.
FR-CR-01	P0	Users may record hypothetical entry, exit, invalidation, sizing, and review criteria independently of executable rules.
FR-BT-01	P0	Run reproducible historical evaluations with point-in-time inputs, benchmark, costs, slippage, execution timing, and a complete configuration hash.
FR-BT-02	P0	Display total return, annualized return when valid, volatility, maximum drawdown, turnover, trade count, win/loss statistics, and benchmark-relative results.
FR-PP-01	P0	Create paper portfolios with currency, starting cash, benchmark, valuation policy, cost-basis method, and inception date.
FR-PP-02	P0	Support paper-only market-next-open and limit orders; lifecycle: draft, submitted, accepted, partially filled, filled, rejected, canceled, expired.

FR-PP-03	P0	Every submission requires an explicit confirmation showing symbol, side, quantity, paper label, assumed execution rule, and estimated cash impact.
FR-PP-04	P0	Maintain immutable fills/transactions and a double-entry-style cash/security ledger; corrections use reversing entries, never destructive edits.
FR-PP-05	P0	Maintain positions, FIFO lots initially, cost basis, realized/unrealized P&L, cash, distributions, valuation snapshots, and reconciliation status.
FR-PP-06	P0	A paper order may reference a signal, rule/model version, or user criterion, but references never initiate the order.
FR-PP-07	P0	Recompute positions and cash from the ledger and flag any mismatch with cached balances.
FR-UX-01	P0	Provide dashboard, watchlists, ETF detail, signal history/evidence, criteria, paper portfolio, orders, transactions, performance, jobs, rules, and settings.
FR-UX-02	P0	Display provider, last completed session, retrieval time, local/UTC timestamp, stale/missing warnings, and hypothetical/non-advisory labels.
FR-OP-01	P0	Operators can manually run, retry, suspend, backfill, and inspect jobs without database editing.
FR-OP-02	P0	Export a redacted diagnostic bundle containing versions, manifests, job summaries, DQ issues, and correlation IDs—never secrets or provider payloads prohibited by terms.
FR-AI-01	P1	Future local AI may summarize existing evidence or draft a constrained rule definition, but cannot execute SQL, provider calls, plugins, or portfolio actions.
FR-EX-01	P1	Export user-owned portfolio, transaction, signal, and run metadata in CSV/JSON subject to provider display/redistribution policy.

5. Market-data assessment and recommendation

No reviewed zero-cost provider simultaneously offers a supported API, broad consolidated U.S. ETF EOD coverage, long history, generous automation, durable storage, and unambiguous organizational redistribution rights.

Provider	Evidence-based assessment	Prototype role
yfinance/Yahoo	Broad Python functionality and fund data, but yfinance is unaffiliated with Yahoo, intended for research/education, and identifies Yahoo Finance API use as personal-only. [ranaroussi.github.io]	Conditional primary for a strictly personal, local prototype after user acceptance of terms and fragility.
Alpha Vantage	Official API; free service states 25 requests/day; supports raw and	Fallback/validation adapter for a small

	split/dividend-adjusted time series. Its terms restrict default rights to private, individual, non-commercial research and treat use on behalf of a corporation as commercial. [alphavantage.co] , [alphavantage.co]	authorized watchlist; never silently mix it into a yfinance dataset.
Tiingo Starter	Technically attractive—500 symbols/month, 50 requests/hour, 1,000/day, 30+ years—but current free-plan terms prohibit durable storage and require organizations to use a commercial plan. [tiingo.com] , [api.tiingo.com]	Reject for this persistent free prototype.
Alpaca Basic	Supported authenticated API, daily bars, adjustment options, and U.S. ETF coverage; however, the unsubscribed feed is IEX-only, which Alpaca describes as appropriate when precise pricing is not primary and approximately 2.5% of market volume. [docs.alpaca.markets] , [docs.alpaca.markets]	Connectivity experiment or secondary comparison—not primary research pricing.
EODHD Free	Authenticated EOD API, adjusted close, 20 calls/day, but free history is limited to the past year and personal use. [eodhd.com] , [eodhd.com]	Fixture/adaptor test only.
Stooq	Downloadable historical datasets, explicitly personal-use-only and commercial use prohibited; no supported automation contract was found. [stooq.com] , [stooq.com]	Manual validation only.

Mandatory provider strategy

1. Implement MarketDataProvider with capabilities: supported assets, date range, raw/adjusted modes, actions, batching, quotas, persistence permission, display permission, and terms URL/version.
2. Do **not** fail over within a dataset. A replacement creates a new provider-specific dataset and an explicit comparison report.
3. Ship deterministic OHLCV fixtures so the application remains buildable and fully testable without accepting third-party terms.
4. Before live ingestion, require a settings acknowledgment that records intended use, provider, terms URL/date, persistence/display permissions, and user choice.

5. If the prototype is used organizationally rather than personally, disable free market-data ingestion until rights are confirmed. This is a product control, not legal advice.
-

6. Economic-data requirements

Use direct official adapters rather than a market-data intermediary:

- **FRED/ALFRED:** primary discovery and vintage-aware economic-series adapter. FRED exposes observations, releases/release dates, update times, vintage dates, and ALFRED real-time periods. [\[fred.stlouisfed.org\]](https://fred.stlouisfed.org)
- **BLS:** labor and price series; the public API serves published historical series, with documented series/query limits and a release calendar. [\[bls.gov\]](https://bls.gov)
- **BEA:** national, regional, international, and industry data with JSON/XML retrieval and metadata over HTTPS. [\[bea.gov\]](https://bea.gov)
- **Treasury Fiscal Data:** debt, revenue, spending, rates, and related federal-finance datasets; the API is open without a token and data is offered free for copying, adaptation, and redistribution. [\[fiscaldata.treasury.gov\]](https://fiscaldata.treasury.gov)

Point-in-time rule: evaluation at time T may use only an observation vintage whose availability/release timestamp is $\leq T$. Store original frequency; any forward-fill, period-end alignment, lag, interpolation, or resampling must be a separately versioned transformation. Never overwrite an economic observation; append vintages and mark the current view.

7. Target architecture

Use **four application deployables plus PostgreSQL**, not a service per table:

1. web-api – **TypeScript:** React/Next.js UI, BFF/REST API, watchlists, settings, signal queries, and user confirmations.
2. portfolio-service – **TypeScript:** paper orders, fills, ledger, lots, valuations, reconciliation, and portfolio calculations.
3. ingestion-worker – **Python:** provider adapters, raw capture, normalization, calendars, market/economic jobs, and DQ.
4. analytics-worker – **Python:** indicators, deterministic rules, backtests, evidence packages, and future plugin adapters.
5. **PostgreSQL:** one cluster with bounded schemas; each service owns writes to its schema.

Use REST/OpenAPI for commands and queries. Use PostgreSQL job/outbox tables and Kubernetes Jobs for asynchronous work; avoid adding Kafka/RabbitMQ for the prototype. Kubernetes CronJobs can create duplicate or missed Jobs, so all jobs must remain idempotent and use concurrencyPolicy: Forbid, a deadline, and explicit UTC time zone. Helm supplies configurable Kubernetes manifests, while ConfigMaps hold non-secret configuration and Secrets hold credentials. [\[kubernetes.io\]](https://kubernetes.io)

The architecture below incorporates the internal provenance-oriented design while reducing its original service count for a greenfield local prototype.

[\[etf_analyt...820_163532 | PDF\]](#), [\[etf_analyt...820_163532 | PDF\]](#)

Figure 1: Local-first architecture: WSL-hosted Kubernetes separates ingestion, analytics, and paper-portfolio accounting while PostgreSQL preserves provenance; user confirmation is the only path from research signals to paper actions, and no brokerage execution exists.

Kubernetes resources

- Namespace, Deployments for the two HTTP services, Jobs/CronJobs for workers, ClusterIP Services, Ingress bound to localhost, ConfigMaps, Secrets, ServiceAccounts, NetworkPolicies, resource requests/limits, PVC for PostgreSQL, migration Job, and PodDisruptionBudgets only if later justified.
- /live, /ready, /health/dependencies; readiness must fail when migrations are pending or required database connectivity is unavailable.
- No LoadBalancer or internet-facing listener by default.

Paper-order state model

```
1 stateDiagram-v2
2     state "Draft" as draft
3     state "Submitted" as submitted
4     state "Accepted" as accepted
5     state "Partially filled" as partial
6     state "Filled" as filled
7     state "Rejected" as rejected
8     state "Canceled" as canceled
9     state "Expired" as expired
10    [*] --> draft
11    draft --> submitted : User confirms
12    submitted --> accepted : Valid
13    submitted --> rejected : Invalid
14    accepted --> partial : Partial fill
```

15	accepted --> filled : Full fill
16	accepted --> canceled : User cancels
17	accepted --> expired : Time expires
18	partial --> filled : Remaining fill
19	partial --> canceled : Cancel remainder
20	filled --> [*]
21	rejected --> [*]
22	canceled --> [*]
23	expired --> [*]

8. PostgreSQL data architecture

Schema	Required entities
catalog	instrument, instrument_symbol, watchlist, watchlist_item, exchange, trading_session
market	provider, provider_policy, ingestion_payload, price_daily, corporate_action, market_dataset, data_revision
economic	source, series, series_version, observation_vintage, transformation
analytics	rule, rule_version, parameters, analytics_run, input_snapshot, indicator_value, signal, evidence
backtest	run, configuration, simulated_order/fill, metric, benchmark_result, warning
portfolio	portfolio, paper_order, paper_fill, transaction, cash_ledger, position, lot, valuation_snapshot, benchmark_snapshot, reconciliation
operations	job_definition, job_run, job_item, data_quality_issue, outbox_event
audit	audit_event, configuration_snapshot, artifact_hash

Persist money, prices, quantities, and ratios as appropriately bounded numeric, not binary floating point. Enforce positive quantity, valid order transitions, unique ingestion keys, immutable fill IDs, nonnegative cash unless explicitly configured, and referential integrity using PostgreSQL check, unique, primary-key, and foreign-key constraints. Do not introduce partitioning until measured volume/query plans justify it; prior internal analysis likewise recommended starting without partitioning. [\[postgresql.org\]](https://www.postgresql.org/) [\[etf_analyt...820_163532 | PDF\]](#)

All migrations are forward-versioned, transactional where supported, checksum-validated, tested against an empty database and a prior release, and executed by a dedicated migration Job before application readiness succeeds.

9. Analytics, plugins, and backtesting

Rule contract

Each rule package must declare:

```
1 rule_id, semantic_version, implementation_hash, runtime,  
2 input schema, output schema, parameters, minimum history,  
3 adjustment policy, calendar, allowed frequencies,  
4 random seed, resource limits, evidence fields, and owner/status
```

P0 rules are deterministic and legacy-inspired only after inspection. Each run receives read-only versioned inputs and returns signals plus structured evidence. Python plugins run in an isolated process/container with no network, no writable host mount, no database credential, read-only inputs, restricted CPU/memory/time, and signed/hash-verified manifests. Rust requires a reproducible benchmark demonstrating material benefit before introduction.

Historical evaluation

- Point-in-time joins; no future observations, revised economic values, or same-day values before availability.
- Configurable commissions, fixed/proportional slippage, execution delay, fractional-share policy, and cash constraints.
- Default market-order research assumption: fill at the next available session open—not the signal close.
- Limit-fill assumptions must be explicit and flagged as OHLC-based estimates.
- Preserve datasets, rule/model version, seed, parameters, code hash, provider, benchmark, and environment.
- Warn about survivorship bias, ETF closure/history limitations, corporate-action quality, multiple testing, and small samples.
- Future ML must use chronological train/validation/test separation and out-of-sample reporting.

10. Paper-portfolio accounting

The portfolio ledger is authoritative; positions and cash balances are derived projections.

- **Orders:** buy/sell, quantity, market-next-open or limit, submitted/expiry dates, user confirmation, signal linkage, status history.
- **Fills:** deterministic simulator version, price source/date, price, quantity, assumptions, fees, slippage, and immutable fill record.

- **Cash ledger:** initial funding, buys, sells, distributions, fees, corrections, and adjustments.
- **Lots:** acquisition fill/date, remaining quantity, unit basis, total basis; FIFO realization in P0.
- **Valuation:** prior-day close, source/revision, cash, market value, basis, realized/unrealized P&L, total equity.
- **Performance:** time-weighted return; money-weighted return only when cash-flow handling is validated; drawdown and benchmark comparison.
- **Reconciliation:** ledger cash equals cached cash; open lots sum to positions; fills sum to order quantities; accounting identity differences equal zero.

No provider-side paper-trading API may be used—the simulation and ledger must remain entirely local.

11. UX and non-functional requirements

- WCAG-oriented keyboard navigation, visible focus, semantic headings, labels, contrast, and non-color-only signal meaning.
 - Responsive for desktop and tablet; core workflows usable at 1280×720.
 - Local fixture target: p95 API response under 1 second for non-analytical requests; dashboard first meaningful content under 2 seconds; targets are quality gates subject to measured adjustment.
 - Every analytical page displays **“Research only — hypothetical — user makes all investment decisions.”**
 - Stale, partial, quarantined, or incompatible datasets use blocking warnings rather than merely colored badges.
 - Destructive actions require confirmation and yield audit events.
 - UI must distinguish trading date, source timestamp, retrieval timestamp, completion timestamp, and display time zone.
-

12. Security, operations, and observability

- Least-privilege PostgreSQL roles per service; non-root containers; read-only root filesystems where feasible; dropped Linux capabilities; pinned base-image digests.
- Secrets excluded from Git, images, logs, diagnostics, and client bundles.
- Dependency, secret, license, and container scanning; GitHub can export SPDX SBOMs and create them through Actions. Generate SBOM and image provenance for release builds. [\[docs.github.com\]](https://docs.github.com)

- Validate all API inputs and symbols; parameterized SQL only; CSRF protection for browser mutations; request/body limits.
 - Nightly local pg_dump with retention setting plus a tested restore command; backup lives outside the database PVC.
 - Structured JSON logs include timestamp, severity, service, version, correlation ID, job/run ID, instrument, provider, and sanitized error code.
 - Instrument TypeScript and Python with OpenTelemetry traces and metrics; Python OpenTelemetry provides metrics, traces, and logs APIs/SDKs, while logging support should be treated according to its documented maturity. [\[opentelemetry.io\]](https://opentelemetry.io)
 - UI operations page exposes last successful session, stale-symbol count, quarantined rows, job duration/status, retries, rule-run history, ledger reconciliation, and dependency health.
-

13. GitHub, IaC, and developer workflow

```
1 /apps/web-api
2 /apps/portfolio-service
3 /workers/ingestion
4 /workers/analytics
5 /packages/contracts
6 /packages/ui
7 /db/migrations
8 /infra/helm
9 /infra/kind
10 /docs/adr
11 /docs/requirements
12 /docs/legacy
13 /tests/fixtures
14 /.github/workflows
```

- Trunk-oriented branches, required PR review, linked requirement IDs, conventional commits, and protected main.
- CI: format, lint, type-check, unit, contract, migration, integration, E2E, Helm render/lint, image build, vulnerability/secret/license scan, SBOM, and kind smoke test.
- Pin GitHub Actions by commit SHA and dependencies with lockfiles; publish versioned images to GHCR.
- make bootstrap, make test, make cluster-up, make deploy, make seed, make diagnostics, make backup, make restore-test, and make teardown.

Windows host responsibilities: enable/update WSL, install Ubuntu and the selected container runtime, allocate resources, and configure localhost/browser access. **WSL Ubuntu responsibilities:** clone into the Linux filesystem, run Git, Node, Python, container, kind, Helm, kubectl, migration, and build commands. Microsoft recommends storing Linux-tool projects in the WSL filesystem rather than /mnt/c for performance. kind runs local Kubernetes using container nodes and is intended for local development or CI. [\[learn.microsoft.com\]](https://learn.microsoft.com) [\[kind.sigs.k8s.io\]](https://kind.sigs.k8s.io)

14. Testing and representative acceptance criteria

Required suites: unit, property/accounting, provider contract, fixture ingestion, migration, API contract, analytics golden-master, time-leakage, backtest, portfolio reconciliation, UI accessibility, E2E, container/kind smoke, source-outage, retry/idempotency, backup/restore, and diagnostic-redaction tests.

1. **Given** an ETF/date already loaded, **when** the same ingestion job reruns, **then** no duplicate current price exists and the rerun is auditable.
 2. **Given** an expected session with missing data, **when** analytics starts, **then** the instrument is skipped, a DQ issue is visible, and no valid signal is published.
 3. **Given** an economic value revised after evaluation date T, **when** a backtest runs as of T, **then** it uses only the vintage available at or before T.
 4. **Given** a displayed signal, **when** the user takes no action, **then** no paper order or portfolio mutation occurs.
 5. **Given** the user initiates a paper order, **when** confirmation is not completed, **then** the order remains draft.
 6. **Given** confirmed starting cash and fills, **when** the portfolio is rebuilt from transactions, **then** cash, lots, positions, realized P&L, and cached projections reconcile exactly within configured decimal precision.
 7. **Given** identical data snapshot, code hash, parameters, environment, and seed, **when** an analytics/backtest run is repeated, **then** outputs and evidence hashes match.
 8. **Given** a provider outage, **when** retries are exhausted, **then** the job is failed—not successful-with-zero-rows—and dependent research is blocked.
 9. **Given** a repository checkout on supported prerequisites, **when** bootstrap and deploy run, **then** all pods become ready, migrations apply once, fixture data loads, and the browser vertical slice works.
 10. **Given** a diagnostic export, **when** inspected, **then** it contains no API keys, passwords, prohibited raw provider data, or brokerage artifacts.
-

15. Legacy-analysis prerequisite

Before final mappings or rule implementation, the build agent must inspect the supplied SQL Server schema/data and .NET solution and produce:

- project/dependency/scheduler inventory;
- table, key, index, procedure, function, view, job, and data-volume inventory;
- symbol/calendar, adjusted-price, null, rounding, timezone, and correction semantics;
- rule formulas, lookback windows, parameters, ordering, state, and output examples;
- sample input/output golden masters;
- SQL Server-to-PostgreSQL mapping with accepted intentional differences;
- legacy-to-target-traceability.csv;
- ADRs for contested mappings.

The agent must not infer equivalence from similar names. Existing internal documentation explicitly labels reconstructed daily workflow assumptions as requiring source validation and calls for machine-readable contracts, mappings, tests, and traceability. [\[etf_analyt...820_163532 | PDF\]](#), [\[etf_analyt...820_163532 | PDF\]](#)

16. Prioritized backlog and build sequence

1. **P0 foundation:** inspect legacy artifacts; freeze contracts; create monorepo, CI, kind, Helm, PostgreSQL, migrations, fixtures, telemetry.
 2. **P0 vertical slice:** watchlist → fixture/provider adapter → normalized daily price → one inspected legacy-inspired rule → evidence-bearing signal → explicit paper order → fill → ledger → dashboard.
 3. **P0 ingestion hardening:** calendars, backfill, idempotency, revisions, DQ quarantine, economic adapter, job UI.
 4. **P0 portfolio completeness:** orders, lots, cash, P&L, valuations, benchmark, reconciliation, performance history.
 5. **P0 backtesting:** point-in-time datasets, costs/slippage, reproducibility, bias warnings, benchmark.
 6. **P0 operational closure:** accessibility, security scans, backup/restore, diagnostics, failure/recovery acceptance.
 7. **P1:** richer charting, multiple rule templates, criteria UI, imports/exports, corporate-action cash handling, provider comparison.
 8. **Future:** local ML/AI, Azure deployment, external identity, event broker, larger universe, streaming, paid/licensed providers, Rust optimization.
-

17. ADR-style decisions, risks, and definition of done

ADR	Decision	Rationale / uncertainty
ADR-001	yfinance conditional personal-primary; Alpha Vantage fallback adapter; fixtures always available	No reviewed free source satisfies every legal and technical requirement. Revalidate terms before use.
ADR-002	PostgreSQL	Relational integrity, transactions, numeric types, portability, and free local operation.
ADR-003	kind on WSL Ubuntu + Helm	Matches required Kubernetes delivery while remaining local and reproducible.
ADR-004	Four deployables, shared PostgreSQL cluster	Preserves bounded responsibilities without prototype microservice sprawl.
ADR-005	TypeScript for browser/API/portfolio; Python for data/analytics	Aligns language strengths; prevents unnecessary polyglot complexity.
ADR-006	REST/OpenAPI + PostgreSQL job/outbox	Simpler than introducing a broker; replacement seam remains available.
ADR-007	Deterministic rule contract first	Supports legacy characterization, evidence, reruns, and later ML adapters.
ADR-008	Append-only portfolio ledger	Enables reconstruction, correction, auditability, and reconciliation.
ADR-009	GitHub Actions + Helm + GHCR	Repeatable CI/CD and Kubernetes artifacts without requiring Azure resources.
ADR-010	Rust only after approved benchmark	Avoids unsupported complexity and premature optimization.

Top risks: provider rights or API changes; adjusted-price revisions; incomplete corporate actions; legacy-rule ambiguity; future-data leakage; ledger defects; WSL/container networking; plugin escape; microservice over-fragmentation; and false confidence from hypothetical results.

Definition of done

The prototype is done when a clean checkout can be bootstrapped in WSL; CI passes; Kubernetes becomes healthy; migrations and fixture/live-authorized ingestion work; a watchlist can be maintained; stale/gap states are visible; at least one inspected legacy-inspired rule produces reproducible evidence; a user can explicitly create and confirm a paper order; fills, ledger, FIFO lots, cash, P&L, valuations, performance, and benchmark reconcile; backtests enforce point-in-time inputs; no brokerage capability exists; backup/restore and diagnostic export pass; and all P0 requirements have automated acceptance evidence and traceability.